4-bit Array Multiplier: RTL to GDSII

ECE463 Digital VLSI Design | Team Assignment Report

Monishram Selvaraj 2024102076
Neel Rajesh 2024102007

Repository: `https://github.com/monishram2508/mult4-rtl-to-gds`

October 2026

1 Summary

A 4-bit unsigned array multiplier was written in Verilog, verified exhaustively, synthesised to SkyWater SKY130 standard cells, timed with static timing analysis, and taken through place and route to a DRC- and LVS-clean GDSII layout.

Part	Result
A: RTL and verification	`PASS: 256 vectors checked, 0 errors` (RTL and gate-level netlist)
B: Logic synthesis	67 cells, of which 16 flip-flops; total area 746.97 μm^2
C: Static timing	$T_{min} = 2.604$ ns, $F_{max} = 384.0$ MHz (typical corner)
D: Physical design	First run passed at 3.5 ns: 0 DRC, 0 LVS, setup slack +0.605 ns, post-route $F_{max} = 345.4$ MHz

Environment. IIC-OSIC-TOOLS Docker image; PDK `sky130A`; standard cell library `sky130_fd_sc_hd`; typical corner `tt_025C_1v80` for Parts B and C. Tools: Icarus Verilog and GTKWave (A), Yosys (B), OpenSTA (C), LibreLane with OpenROAD, Magic, KLayout and Netgen (D). The exact command for every step is listed in the repository README.

2 Part A: RTL Design and Verification

2.1 Design

Port	Direction	Width	Description
`clk`	input	1	Clock, rising edge
`rst_n`	input	1	Asynchronous reset, active low; clears all registers
`a`	input	4	Multiplicand
`b`	input	4	Multiplier
`p`	output	8	Product $a \times b$, registered

`rtl/mult_array.v` has three layers:

1. **Input registers.** `a` and `b` are captured into `a_reg` and `b_reg` (4 + 4 flip-flops) on every rising edge.
2. **Combinational array,** built from the registered values: 16 AND gates form the partial products $pp_i[j] = a_reg[j] \cdot b_reg[i]$, and 12 instances of `full_adder` (`rtl/full_adder.v`, written

with `assign` statements) add them in three ripple-carry rows, instantiated with `generate` loops. The `*` operator is not used.

3. **Output register.** The 8-bit result is captured into `p` (8 flip-flops).

This gives exactly 16 flip-flops and a latency of 2 clock cycles: inputs are captured at edge 1 and the product at edge 2. The array is wired exactly as the connection table in Section 2 of the assignment:

Row	Inputs to full adder j ($j = 0 \ldots 3$)	Outputs
0	no adders	$p[0] = pp_0[0]$; $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
1	$pp_1[j]$, $r_0[j]$, $c_1[j]$ $(c_1[0] = 0)$	$p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3], s_1[2], s_1[1]\}$
2	$pp_2[j]$, $r_1[j]$, $c_2[j]$ $(c_2[0] = 0)$	$p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3], s_2[2], s_2[1]\}$
3	$pp_3[j]$, $r_2[j]$, $c_3[j]$ $(c_3[0] = 0)$	$p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$

2.2 Block diagram

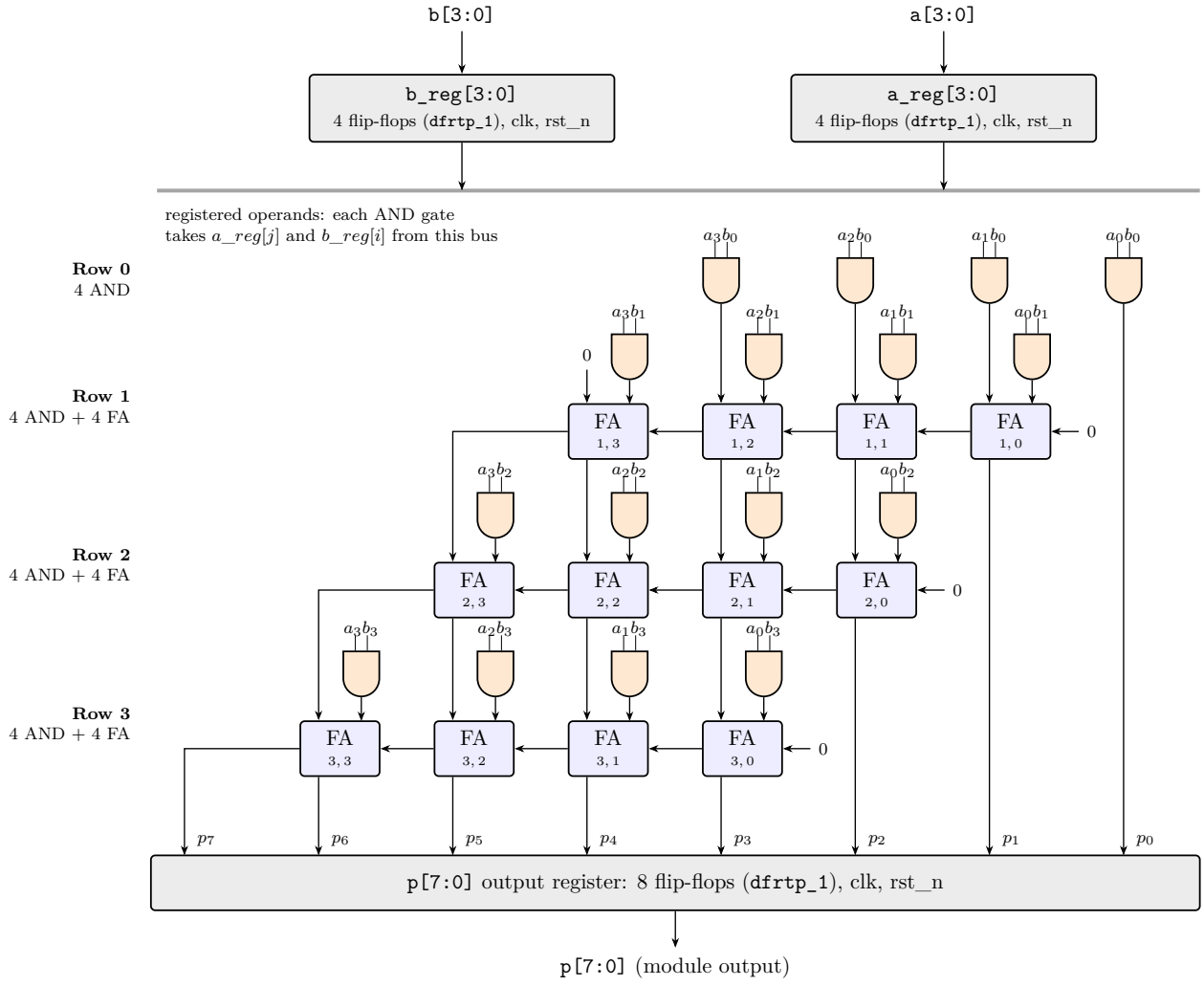


Figure 1: Block diagram: 8 input flip-flops, 16 AND gates, 12 full adders (FA i, j is adder j of row i) and 8 output flip-flops. Columns are aligned by bit weight, weight 0 on the right. Each FA takes the partial product from the AND gate above it (right input) and the bit passed down from the previous row (left input); the carry enters from the right and leaves to the left, and the sum leaves from the bottom.

2.3 Testbench

`tb/tb_mult.v` is exhaustive and self-checking:

- a 10 ns clock, with `rst_n` held low for the first 2 cycles;
- all 256 combinations $a, b \in \{0 \dots 15\}$, applied on the falling edge;
- for each one it waits the 2-cycle latency (two rising edges), then samples `p` on the next falling edge and compares it with `a * b` (the `*` operator appears only in the testbench);
- it prints a line for every mismatch, exactly one summary line at the end, and writes `mult.vcd`.

Sampling on the falling edge keeps the check away from the clock edge. This matters for the gate-level simulation in Part B, where every cell has a unit delay and a sample taken right after the rising edge would race with the flip-flop outputs.

2.4 Results

Simulation of the RTL:

```
PASS: 256 vectors checked, 0 errors
```

Error test (task 6). `sum` in `full_adder.v` was changed from `a ^ b ^ cin` to `a ^ b`; the testbench caught it (first mismatches and the summary line shown), and the change was then undone:

```
MISMATCH: a=3 b=3 expected=9 (00001001) got=5 (00000101) at time 1066000
MISMATCH: a=3 b=6 expected=18 (00010010) got=10 (00001010) at time 1126000
MISMATCH: a=3 b=7 expected=21 (00010101) got=9 (00001001) at time 1146000
FAIL: 256 vectors checked, 95 errors
```

2.5 Worked example (task 5)

A = last digit of Neel Rajesh's roll number (2024102007) $+5 = 7 + 5 = 12$. Since $A \neq 7$, $B = 7$. Expected product: $12 \times 7 = 84 = 0101\,0100_2$.

Worked in the same layout as the example in Section 2 of the assignment: one partial product $pp_i = A$ AND b_i per bit of B, shifted left by i, then added.

`1 1 0 0`	A	12
`× 0 1 1 1`	B	7
`1 1 0 0`	$pp_0 = A$ AND b_0 ($b_0 = 1$)	12
`1 1 0 0`	$pp_1 = A$ AND b_1 ($b_1 = 1$), shifted 1	24
`1 1 0 0`	$pp_2 = A$ AND b_2 ($b_2 = 1$), shifted 2	48
`0 0 0 0`	$pp_3 = A$ AND b_3 ($b_3 = 0$), shifted 3	0
`0 1 0 1 0 1 0 0`	$P = 12 + 24 + 48 + 0$	**84**

Table 1: Worked example $12 \times 7 = 84$.

The same product traced through the array, row by row, using the connection equations of Section 2 (r_i is the 4-bit value row i passes down; s_i and $c_i[4]$ are the sum bits and final carry of row i):

Row	pp_i	Addition	Carry, sum	Product bit	Passes down
0	1100	AND gates only	–	$p[0] = pp_0[0] = 0$	$r_0 = 0110$
1	1100	$pp_1 + r_0 = 12 + 6 = 18$	$c_1[4] = 1$, $s_1 = 0010$	$p[1] = s_1[0] = 0$	$r_1 = 1001$
2	1100	$pp_2 + r_1 = 12 + 9 = 21$	$c_2[4] = 1$, $s_2 = 0101$	$p[2] = s_2[0] = 1$	$r_2 = 1010$
3	0000	$pp_3 + r_2 = 0 + 10 = 10$	$c_3[4] = 0$, $s_3 = 1010$	$p[3] = s_3[0] = 0$	$p[7:4] = 0101$

Table 2: 12×7 through the array. Reading $p[7:4]$ and $p[3:0]$ together gives $p = 0101\,0100 = 84$.

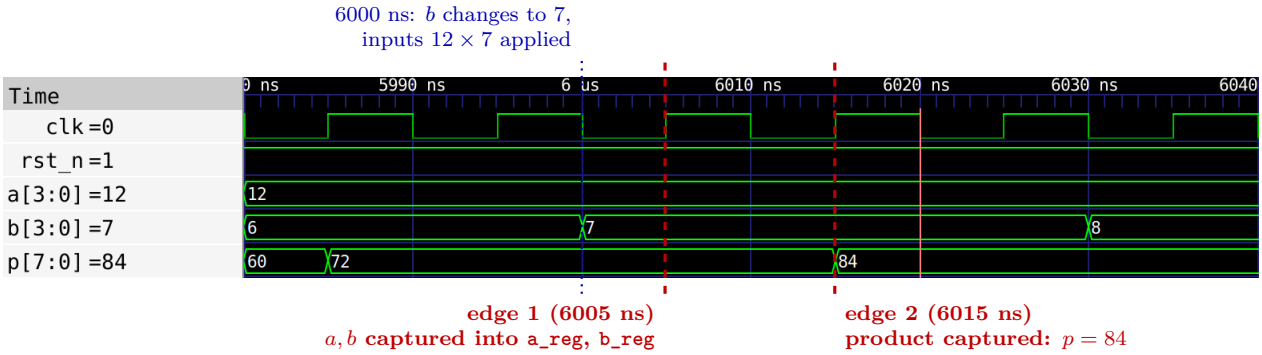


Figure 2: RTL waveform (GTKWave, 5980–6040 ns) for 12×7. The inputs applied before edge 1 are captured at edge 1, and the product reaches `p` at edge 2: a latency of 2 clock cycles. The GTKWave marker at 6020 ns, where the testbench samples `p`, gives the values shown next to the signal names.

Time (ns)	Event	a	b	p
5985	rising edge	12	6	60 → 72 (result of 12×6)
6000	new inputs applied (falling edge)	12	7	72
6005	rising edge 1: inputs captured	12	7	72
6015	rising edge 2: product captured	12	7	**84**
6030	next inputs applied	12	8	84

Table 3: Values read from Figure 2.

3 Part B: Logic Synthesis with Yosys

`synth/synth.tcl` runs the seven required steps (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat`, `write_verilog`) against the typical-corner Liberty file.

Metric	Value
Total number of cells	67
Number of flip-flops	16
Total cell area	746.966 μm^2
Flip-flop area	400.384 μm^2 (16×25.024)
Flip-flop area (% of total)	53.60%
3 most-used cell types	`dfrtp_1` (16), `nand2_1` (12), `xnor2_1` (7)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 4: Part B results.

Cell	Count	Area (μm^2)
dfrtp_1	16	400.384
nand2_1	12	45.043
xnor2_1	7	61.309
a21oi_1	4	20.019
and2_0	2	12.512
lpflow_isobufsrc_1	2	12.512
maj3_1	2	20.019
nand2b_1	2	12.512
nand4_1	2	12.512
nor3_1	2	10.010
o21a_1	2	15.014
o21ai_0	2	10.010

Cell	Count	Area (μm^2)
xnor3_1	2	45.043
a21boi_0	1	7.507
a22oi_1	1	7.507
a32oi_1	1	8.758
and4_1	1	8.758
clkinv_1	1	3.754
nand3_1	1	5.005
nor2_1	1	3.754
o21ba_1	1	10.010
or3_1	1	6.256
xor2_1	1	8.758
Total	**67**	**746.966**

Table 5: All cells in the synthesised netlist (`synth/mult_array_stat.rpt`).

The synthesised netlist was simulated with the same testbench, using the SKY130 cell models with `-DFUNCTIONAL -DUNIT_DELAY=#1`, and printed `PASS: 256 vectors checked, 0 errors`, so synthesis did not change the behaviour.

4 Part C: Static Timing Analysis with OpenSTA

`sta/sta.tcl` reads the typical-corner Liberty file and the netlist, defines the clock, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path. At $T = 10$ ns the worst slack was 7.396 ns, so

$$T_{min} = 10 - 7.396 = 2.604 \text{ ns}, \qquad F_{max} = \frac{1000}{2.604} = 384.0 \text{ MHz}.$$

Re-running at $T = 2.60$ ns gave a slack of -0.004 ns, close to 0, which confirms the calculation.

Metric	Value
Worst slack at $T = 10$ ns	7.396 ns
T_{min}	2.604 ns
F_{max}	384.0 MHz
Slack at $T = T_{min}$ (2.60 ns)	-0.004 ns (≈ 0)
Critical path start point	`_106_` (`dfrtp_1`, `b_reg[0]`)
Critical path end point	`_100_` (`dfrtp_1`, `p[6]`)
Cells on the critical path	8 combinational (10 including both flip-flops)
Latency $= 2 \times T_{min}$	5.208 ns
Throughput	384.0 million results per second

Table 6: Part C results.

#	Instance	Cell	Delay (ns)	Arrival (ns)	Function
–	_106_	dfrtp_1	0.388	0.388	launch flip-flop b_reg[0] (clock to Q)
1	_043_	and2_0	0.180	0.568	partial product (AND)
2	_056_	or3_1	0.391	0.959	carry logic merged by ABC
3	_058_	a32oi_1	0.250	1.208	partial products merged into carry logic
4	_072_	o21ba_1	0.198	1.406	carry logic merged by ABC
5	_073_	xnor2_1	0.147	1.553	adder sum
6	_082_	maj3_1	0.385	1.937	full-adder carry (majority)
7	_084_	maj3_1	0.358	2.295	full-adder carry (majority)
8	_088_	xnor3_1	0.175	2.471	full-adder sum, drives p_next[6]
–	_100_	dfrtp_1	setup 0.134	–	capture flip-flop p[6]

Table 7: Critical path at $T = 2.60$ ns (sta/mult_array_timing.rpt).

Cells from full adders (task 3). Four of the eight combinational cells are plain full-adder functions: two maj3_1 carries, one xnor3_1 sum and one xnor2_1 sum. One, and2_0, is a partial-product AND gate. The other three (or3_1, a32oi_1, o21ba_1) are compound cells in which ABC merged partial-product ANDs into the carry logic. Section 2 estimates the longest path as about $2N = 8$ full-adder delays. The path does have 8 cells, but each one is a single library cell, not a full adder built from two gate levels (see question 2).

5 Part D: Physical Design with LibreLane

Key	Value
DESIGN_NAME	"mult_array"
VERILOG_FILES	["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"]
CLOCK_PORT	"clk"
CLOCK_PERIOD	3.5 ($1.2 \times T_{min} = 1.2 \times 2.604 = 3.125$, rounded up to the next 0.5 ns)
FP_SIZING	"absolute"
DIE_AREA	[0, 0, 150, 150]

Table 8: pnr/config.json.

The first run (RUN_2026-10-25_10-24-37) finished with 0 DRC, 0 LVS and 0 antenna errors and positive setup slack, so no further runs were needed. final/metrics.json and final/gds/mult_array.gds were copied into pnr/.

Metric	Value
`CLOCK_PERIOD` used	3.5 ns
Number of runs needed	1
Core area	17 759.5 μm^2 (die $150 \times 150 = 22\,500$ μm^2)
Cells after synthesis (Part B)	67
Cells after place and route	1886
Kinds of cells added	decap, fill, tap, I/O buffers, clock buffers, timing-repair buffers
Wirelength	2381 μm
Setup slack	+0.605 ns (typical corner, `nom_tt_025C_1v80`)
Post-route F_{max}	$1000/(3.5 - 0.605) = 345.4$ MHz
F_{max} from Part C	384.0 MHz
DRC errors	0 (Magic 0, KLayout 0)
LVS errors	0

Table 9: Part D results (`pnr/metrics.json`).

Kind	Cells (instance names)	Count
Decap	`decap_3`, `decap_4`, `decap_6`, `decap_8`, `decap_12`	1292
Fill	`fill_1`, `fill_2`	258
Tap	`tapvpwrvgnd_1`	245
Input/output buffers	`buf_2` (`input1`–`input9`, `output10`–`output17`)	17
Clock tree buffers	`clkbuf_16`	3
Setup-repair buffers	`buf_4` (`rebuffer*`)	5
Hold-fix delay cells	`dlygate4sd3_1` (`hold*`)	3
Fanout buffers	`clkdlybuf4s50_1` (`fanout*`)	2
Logic		45
Flip-flops	`dfrtp_1`	16
Total		**1886**

Table 10: Where the 1886 cells come from (`metrics.json` and the final netlist). LibreLane runs its own Yosys synthesis from the RTL, which is why the logic is 45 cells here and 51 in Part B.

Timing corner. Parts B and C use the typical corner, so the slack quoted above is the typical-corner value, which keeps the comparison with Part C like for like. LibreLane also analyses fast and slow corners. All fast and typical corners pass setup; the slow corner (`ss_100C_1v60`, 1.60 V and 100 °C) does not. The all-corner `timing__setup__ws` in `metrics.json` is −2.118 ns, with 5 violations in each slow corner. Hold slack is positive at all nine corners (worst +0.104 ns).

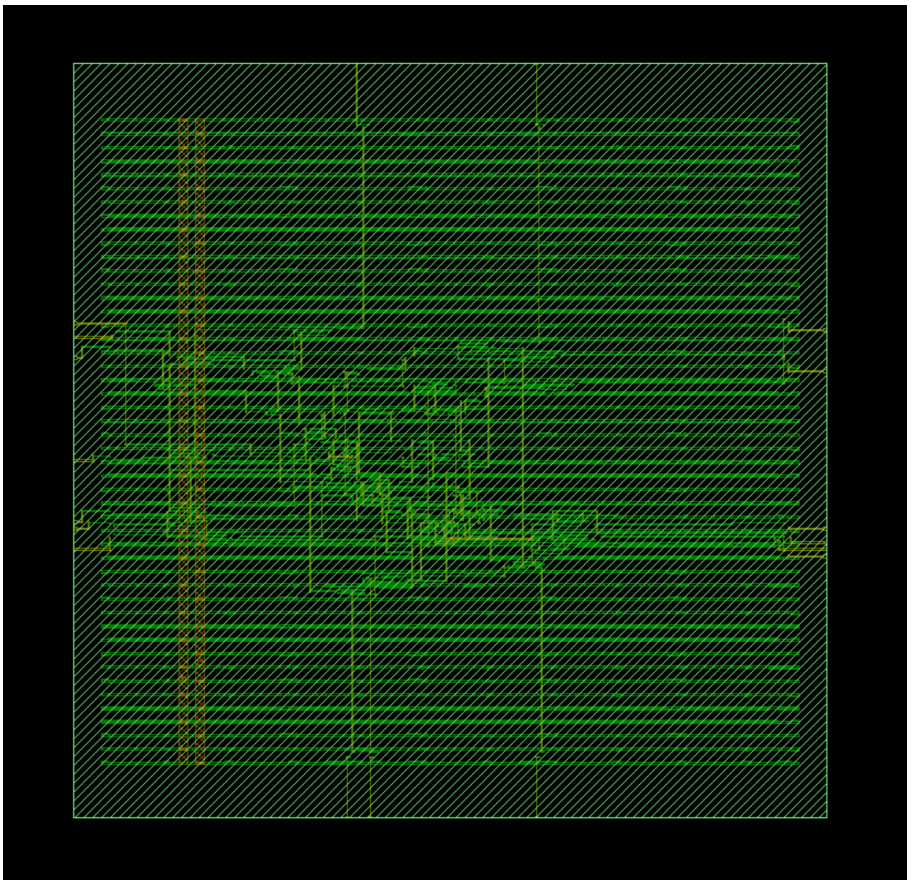


Figure 3: Whole die in KLayout, metal layers 1–4 only. The horizontal lines are the metal-1 power rails, one per standard-cell row; the vertical pair on the left is a metal-4 power strap. The signal routing is concentrated in the centre, where the multiplier logic sits, with wires running out to the pins on the left and right edges.

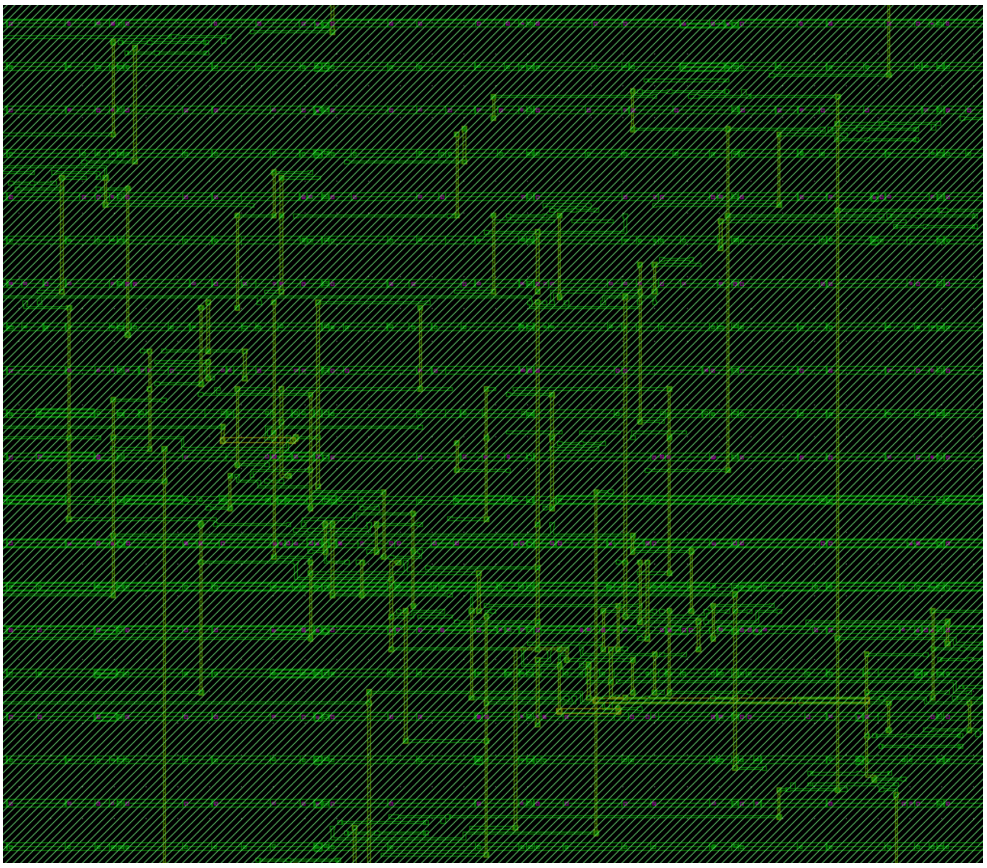


Figure 4: Zoomed view of the centre of the die, metal layers only. Each horizontal line is a metal-1 power rail along a standard-cell row; the wires between them are the signal routing on metal 1–3.

6 Answers to the Questions

1. Logic cells versus 16 AND gates + 12 full adders

Synthesis produced 67 cells, of which 16 are flip-flops, so $67 - 16 = 51$ cells implement the logic, against the $16 + 12 = 28$ components in the RTL. The numbers differ because synthesis does not keep the RTL structure; it maps the Boolean functions of the whole flattened array onto the cells in the library:

- **A full adder is not one cell.** SKY130 has a full-adder cell, but it has two outputs, and ABC maps single-output functions only. So each sum and each carry is built separately: the sums from XOR/XNOR cells (7 `xnor2_1`, 2 `xnor3_1`, 1 `xor2_1`), the carries from majority and AND-OR-invert cells (2 `maj3_1`, plus `a21oi`, `a22oi`, `a32oi`, `o21a`, `o21ai` and others).
- **The AND gates mostly disappear.** Only 2 `and2_0` and one `and4_1` remain; the other partial products are absorbed into the 12 `nand2_1` cells and into compound AOI/OAI cells.
- **Constants simplify the array.** $c_1[0] = c_2[0] = c_3[0] = 0$ and $r_0[3] = 0$ turn four of the full adders into half adders, and Yosys removed the constant top bit of r_0 (`WREDUCE: Removed top 1 bits (of 4) from wire mult_array.r0`).
- **Polarity fixes.** A few cells (`clkinv_1`, `nand2b_1`, `lpflow_isobufsrc_1`) exist only to invert or gate signals so that the inverting cells above can be used.

2. Critical path

The critical path runs from flip-flop `_106_` (`b_reg[0]`) to flip-flop `_100_` (`p[6]`), through 8 combinational cells (10 including the two flip-flops), with 2.083 ns of logic delay after the 0.388 ns clock-to-Q. Its shape matches what Section 2 predicts: it starts at an operand bit that feeds the first rows and ripples through the carry chains down to a high product bit. Section 2 estimates about $2N = 8$ full-adder delays. The path does have 8 cells, but they are not 8 full adders: only 4 are plain full-adder functions (two `maj3_1` carries, one `xnor3_1` and one `xnor2_1` sum), one is a partial-product AND, and three are compound cells in which ABC merged ANDs into the carry logic. Because SKY130 has one-cell majority and three-input XNOR gates, a full-adder carry or sum costs one cell (about 0.26 ns on average here) instead of two gate levels. ABC also restructured the logic across row boundaries, so the real path is shorter than 8 two-level full adders would be.

3. Flip-flop share of the area

Flip-flops take 400.384 of 746.966 μm^2, which is **53.60%**. A `dfrtp_1` is 25.024 μm^2, 6.7 times a `nand2_1` (3.754 μm^2). It is a master-slave flip-flop with an asynchronous reset: the LVS report shows 28 transistors for `dfrtp_1` against 4 for `nand2_1`. The design needs 16 of them (4 + 4 input bits and 8 output bits), while the arithmetic is only 51 small gates. Register count grows linearly with the width ($4N$) but the array grows with N^2, so in a multiplier this small the registers dominate. At $N = 8$ the array would be about 4.7 times larger (56 instead of 12 full adders) while the registers would only double, so the flip-flop share would drop.

4. Why post-route F_{max} is lower, and the added cells

Post-route F_{max} is $1000/(3.5 - 0.605) = 1000/2.895 = 345.4$ MHz, 10% below the 384.0 MHz of Part C. Part C timed an unplaced netlist with an ideal clock (zero delay and zero skew), no extracted wire parasitics and no margin. Post-route timing uses the extracted wire resistance and capacitance, the real clock tree, the buffers the flow inserted, and LibreLane's own constraints. Both analyses report the same critical path, `b_reg[0]` $\rightarrow$ `p[6]`, so the difference in minimum period ($2.895 - 2.604 = 0.291$ ns) can be split exactly:

Contribution	ns
Clock uncertainty: LibreLane's constraints reserve 0.25 ns for clock jitter; Part C had none	+0.250
Data path, clock-to-Q to D: 2.522 vs 2.471 ns (real wire load, two inserted `buf_4` rebuffers, different logic from LibreLane's synthesis)	+0.051
Clock tree: the launch clock arrives 0.030 ns later than the capture clock, offset by 0.031 ns of common-path pessimism removal	−0.001
Setup time of the capture flip-flop: 0.124 vs 0.134 ns	−0.010
Total (difference from rounding)	+0.290

So in this design most of the loss is the clock-uncertainty margin. The physical effects add only about 0.05 ns, because the wires are short: 2381 μm in total, and the longest net is 151 μm. In a larger design, wire delay would be a much bigger share.

The flow added these kinds of cells (Table 10):

- **Decap (1292):** capacitors between the supply rails that deliver charge locally when many cells switch at once, reducing supply droop and noise.
- **Fill (258):** close the gaps in each row so that the wells, implants and power rails are continuous, which the manufacturing (DRC and density) rules require.

- **Tap (245):** connect the n-well to VPWR and the substrate to VGND at regular intervals to prevent latch-up; `sky130_fd_sc_hd` cells have no taps of their own.
- **Input/output buffers (17):** one on each input pin except the clock and one on each output, isolating the internal logic from external loads and drivers.
- **Clock tree buffers (3):** clock tree synthesis distributes `clk` to all 16 flip-flops with small skew and clean edges.
- **Setup-repair buffers (5):** the resizer splits and strengthens slow, heavily loaded nets. Two of them are on the critical path.
- **Hold-fix delay cells (3):** the shortest paths, `a_reg[0]` and `b_reg[0]` through a single AND gate to `p[0]`, would change too soon after the clock edge, so delay cells were added to them.
- **Fanout buffers (2):** `rst_n` drives all 16 flip-flop resets, so it was split into two nets driving 10 and 6 flip-flops.

5. DRC and LVS

DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures and densities on every layer. **LVS** (layout versus schematic) extracts the circuit from the layout and checks that it is the same circuit as the intended netlist: the same devices, connected in the same way. For our layout, Magic and KLayout both reported 0 DRC errors; for LVS, both netlists contained 1886 cell instances and 103 nets, and Netgen reported *Circuits match uniquely.* Both must pass because each catches what the other cannot. A DRC-clean layout can still be the wrong circuit (a misconnected or missing wire), and an LVS-clean layout can still be unmanufacturable (shapes too close together become shorts, or too thin become opens, on silicon). Only a layout that passes both is both buildable and correct.

6. Scaling to 8 bits

Using the formulas in Section 2 with $N = 8$ instead of $N = 4$:

$$\begin{aligned}
\text{AND gates:}\quad & N^2 = 16 \;\rightarrow\; 64 \quad (4\times) \\
\text{Full adders:}\quad & N(N-1) = 4 \times 3 = 12 \;\rightarrow\; 8 \times 7 = 56 \quad (4.7\times) \\
\text{Critical path:}\quad & 2N = 8 \;\rightarrow\; 16 \text{ full-adder delays} \quad (2\times) \\
\text{Flip-flops:}\quad & 4N = 16 \;\rightarrow\; 32 \quad (2\times)
\end{aligned}$$

Our 4-bit path has 2.083 ns of logic for 8 full-adder delays, about 0.26 ns each. For 8 bits:

$$T_{min} \approx 16 \times 0.26 + 0.388 + 0.134 = 4.17 + 0.52 = 4.69 \text{ ns}, \qquad F_{max} \approx \frac{1000}{4.69} \approx 213 \text{ MHz}.$$

The full-adder count grows with N^2 but the delay only with N: doubling the width roughly quadruples the logic and doubles the critical-path delay, so F_{max} falls from 384 MHz to about 213 MHz.